



Università degli Studi di Genova

Corso di Laurea in Informatica (L-31)

Valutazione di Metriche per il Rilevamento di Attacchi al Codice Sorgente

Relatore:

Prof. Giovanni Lagorio

Co-Relatori:

Dott. Giacomo Benedetti

Dott. Luca Caviglione

Candidato:

Marco Zoratti

Anno Accademico 2024-2025

Indice

1	Introduzione	1
2	Background	2
2.1	Attacchi BST	2
2.2	Attacchi HUT	3
2.3	Gap da Colmare	5
3	Approccio Metodologico	5
3.1	Costruzione di un Dataset contenente Vulnerabilità	7
3.1.1	Raccolta	7
3.1.2	Iniezione della Backdoor	7
3.2	Metodologia	8
4	Studio delle Metriche	8
4.1	Analisi e Invalidazione delle Metriche Generiche	9
4.2	Sviluppo e Validazione di Metriche Specifiche	10
5	Sviluppo della Soluzione	12
5.1	Architettura del Software	13
5.2	Sistema di Configurazione Centralizzato	13
5.3	Componenti di Analisi Implementati	14
5.4	Ottimizzazione delle Prestazioni	14
5.5	Automazione del Processo di Analisi	16
6	Risultati	16
6.1	Efficacia nel Rilevamento degli Attacchi BST	16
6.2	Efficacia nel Rilevamento degli Attacchi HUT	19
6.3	Analisi dei Falsi Positivi	20
7	Discussione	21
7.1	Interpretazione dei Risultati	21
7.2	Fragilità dell'Ecosistema di Sviluppo	21
7.3	Contributi della Tesi	22
7.4	Limiti dello Studio	23
7.5	Lavori Futuri e Implicazioni Pratiche	23
7.6	Conclusioni	24

1 Introduzione

Lo sviluppo del software è legato a un vasto e interconnesso ecosistema di componenti, spesso pubblicati con una licenza open source. La moderna Software Supply Chain (SSC) è una complessa rete di dipendenze che si estende attraverso repository pubblici propri dell’ecosistema di sviluppo come npm (per JavaScript/Node.js), PyPI (Python) e RubyGems (Ruby).

Questo modello, fondato sulla condivisione del codice e sulla fiducia reciproca tra sviluppatori, ha accelerato il processo di creazione delle applicazioni e favorito la nascita di soluzioni e servizi innovativi. Tuttavia, la natura distribuita delle moderne SSC ha influenzato pesantemente la superficie di attacco del software, rendendola più complessa e difficile da definire.

Gli attacchi che modificano il codice sorgente e si nascondono al suo interno sono intrinsecamente insidiosi e sono l’argomento principale affrontato in questa tesi. In questo scenario, l’aggressore non aggira il processo di sviluppo, ma lo sfrutta a proprio vantaggio con l’obiettivo di modificare direttamente il listato in modo che la modifica malevola venga accettata e integrata nel repository principale attraverso i canali standard di contribuzione, come una Pull Request (PR) su *GitHub*. Il codice distribuito sul registro di pacchetti sarà quindi perfettamente identico a quello presente nel repository, poiché la compromissione è avvenuta direttamente a livello del sorgente.

In questo contesto, il processo di revisione del codice assume un ruolo critico. All’interno degli ecosistemi open source, la revisione delle modifiche da parte dei maintainer del progetto o di altri membri della comunità è il principale meccanismo di controllo della qualità.

Quando un potenziale aggressore sottomette una PR, il codice proposto viene visualizzato in un’interfaccia di *diff*, che evidenzia i cambiamenti rispetto alla versione corrente del codice. Il revisore umano ha il compito di esaminare attentamente queste modifiche per assicurarsi che siano sicure, corrette e in linea con gli obiettivi del progetto prima di approvarne l’integrazione.

All’interno di questo processo, è possibile sfruttare la componente umana della catena di sviluppo. Questa tesi si propone di analizzare non una debolezza tecnica, ma la possibilità da parte di un attaccante di “manipolare” la percezione visiva dello sviluppatore. Nello specifico, saranno analizzati due principali famiglie di attacchi:

1. **Blank Space Trojan (BST):** Questa tecnica permette di “appendere” codice malevolo, minificandolo in un’unica riga, al termine di grande quantità di caratteri di spaziatura (spazi o tabulazioni). L’obiettivo è spingere il payload dannoso al di fuori del riquadro di visualizzazione standard di un editor, rendendolo di fatto invisibile al revisore a meno di un deliberato scorrimento orizzontale dell’interfaccia.
2. **Hidden Unicode Trojan (HUT):** Questa categoria di attacchi sfrutta l’enorme quantità di caratteri dello standard Unicode per far sì che il codice sorgente venga interpretato in modo diverso dal compilatore rispetto a come appare al revisore umano. I principali meccanismi studiati in questa tesi sono:
 - **Somiglianza Visiva tramite Omoglifi:** La tecnica consiste nell’usare nei nomi delle funzioni lettere visivamente identiche ma che appartengono ad

alfabeti diversi, come la ‘a’ latina e la ‘a’ cirillica. Mentre l’occhio umano non nota alcuna differenza, il compilatore identifica i caratteri come diversi: questa differenza permette di eseguire una funzione malevola al posto di quella legittima senza che il revisore possa accorgersene.

- **Caratteri di Controllo Invisibili:** La tecnica mira ad inserire nel codice caratteri di controllo che non hanno una rappresentazione grafica ma possono modificare l’ordine di esecuzione delle operazioni, nascondendo comandi malevoli all’interno di codice che a una prima vista sembra innocuo.

Sfruttando i limiti della percezione umana, gli attaccanti riescono a inserire backdoor e codice malevolo che appaiono innocui durante il processo di code review. Investigare questi attacchi è cruciale perché essi minano il fondamento di fiducia dei processi open source, trasformando la revisione umana nella principale superficie d’attacco e richiedendo un nuovo paradigma di sicurezza.

Data l’inefficacia della sola revisione umana, l’obiettivo di questa tesi è l’individuazione e la validazione di alcune metriche descrittive degli artifatti software, capaci di agire come indicatori per gli attacchi BST e HUT. Il contributo del lavoro è quindi nell’avanzamento della sicurezza della Software Supply Chain, in particolare attraverso l’analisi dell’evoluzione storica del codice che costituisce un’applicazione software.

Il resto della tesi è strutturato come segue: la Sezione 2 fornisce il background necessario, illustrando esempi di attacchi reali. La Sezione 3 descrive l’approccio metodologico, dalla creazione del dataset alla definizione della metodologia applicata. La Sezione 4 analizza in dettaglio le metriche software, validando quelle più efficaci. La Sezione 5 descrive l’architettura tecnica del tool sviluppato. Infine, la Sezione 6 presenta e analizza i risultati ottenuti, inclusa una discussione sui falsi positivi.

2 Background

In questa sezione verranno esposti alcuni esempi di attacchi BST (Sezione 2.1) e HUT (Sezione 2.2).

2.1 Attacchi BST

Il primo esempio è relativo al pacchetto *colorama* dell’ecosistema Python, una libreria multi-piattaforma che consente di colorare l’output testuale visualizzato attraverso il terminale. In questo caso, gli attaccanti sono riusciti a distribuire una versione malevola del package grazie ad un attacco di *typosquatting* e codice malevolo nascosto tramite BST, come mostrato dalla Figura 1.

Per prima cosa, gli aggressori hanno creato un sito mirror fasullo per distribuire una versione dannosa del pacchetto. Successivamente, dopo aver compromesso gli account di alcuni sviluppatori, hanno modificato il progetto *top-gg* per far scaricare *colorama* dal loro sito malevolo. Infine, il codice dannoso all’interno del pacchetto era nascosto tramite BST, ovvero celato da una grande quantità di spazi bianchi.

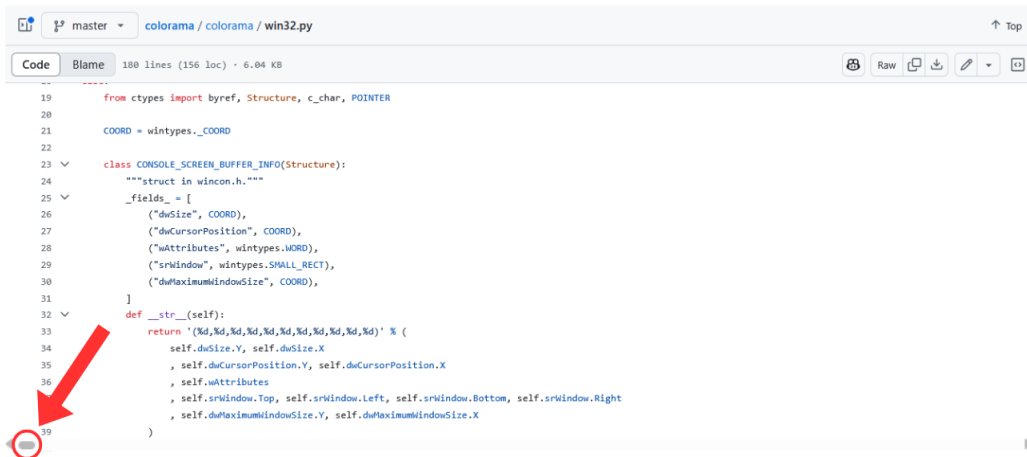


Figura 1: Simulazione dell'attacco BST al package *colorama*. Il codice malevolo è nascosto fuori dalla schermata visibile a causa degli spazi bianchi

Questo caso di studio dimostra come una tecnica di offuscamento visivo relativamente semplice come BST possa essere integrata in un attacco alla SSC, combinando social engineering e typosquatting per massimizzare l'impatto [1].

Il secondo esempio è relativo a un grave incidente di sicurezza che ha colpito il popolare pacchetto *rand-user-agent* presente su npm, utilizzato per distribuire un sofisticato trojan ad accesso remoto.

Gli aggressori hanno rubato gli account degli sviluppatori e pubblicato numerose versioni malevole, nascondendo il codice malevolo tramite BST, come mostrato in Figura 2.

Questo caso esemplifica un attacco all'infrastruttura di deployment, dove il codice sorgente su *GitHub* rimaneva pulito, ma il pacchetto distribuito su npm era compromesso.

La minaccia non è stata scoperta da una revisione umana, ma da uno strumento di scansione automatizzata della sicurezza (Aikido Security [2]) che ha analizzato staticamente il pacchetto e rilevato il codice offuscato, ma non l'occultamento. Successivamente, un intervento umano ha portato alla luce l'attacco perfettamente nascosto e funzionante.

2.2 Attacchi HUT

La prima tipologia di attacchi HUT analizzata sono i *Trojan Source* [5]. Queste minacce fanno sì che l'ordine del codice sorgente appaia diverso al revisore rispetto a come viene letto dal compilatore. A tal fine, l'attacco sfrutta i caratteri Unicode di controllo direzionale, chiamati *BiDi* [12].

La Figura 3 mostra il funzionamento di un attacco *Trojan Source*:

- Aspetto ingannevole (Figura 3a): Il codice appare normale e sicuro all'occhio del revisore umano.
- Contenuto reale (Figura 3b): Il file sorgente nasconde in realtà dei caratteri di controllo invisibili che alterano l'ordine visivo del testo.
- Esecuzione malevola (Figura 3c): Compilatori ed interpreti ignorano i caratteri *BiDi* leggendo il codice nel suo ordine logico e dannoso, eseguendo l'attacco.

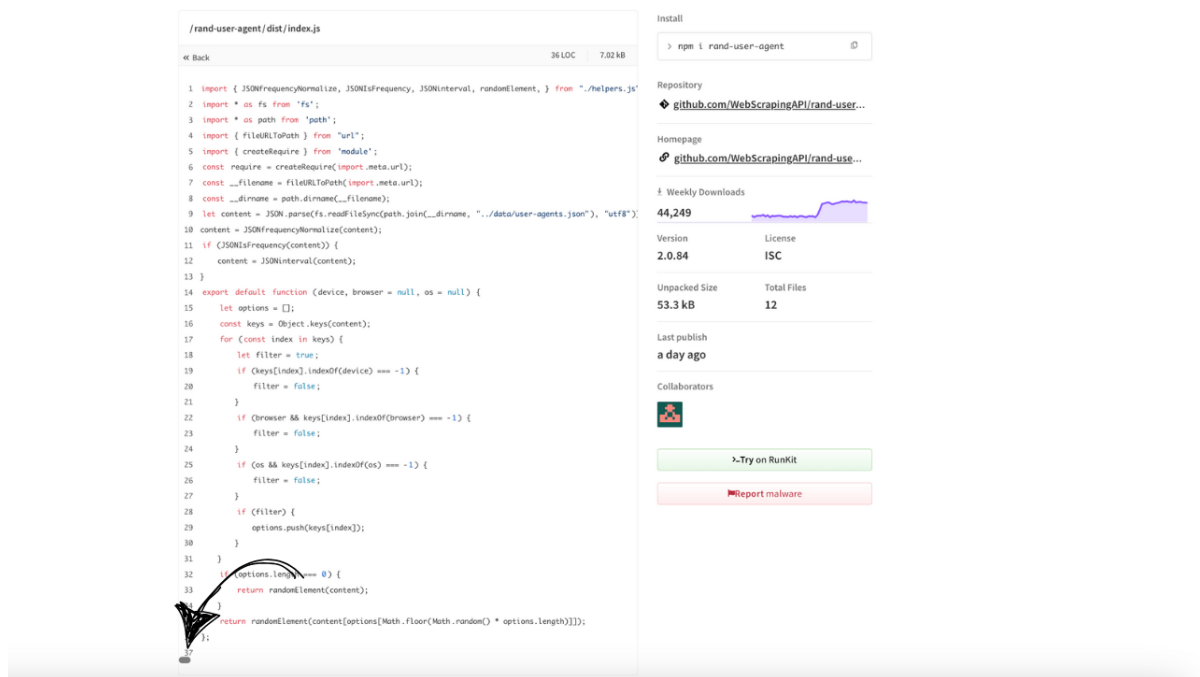
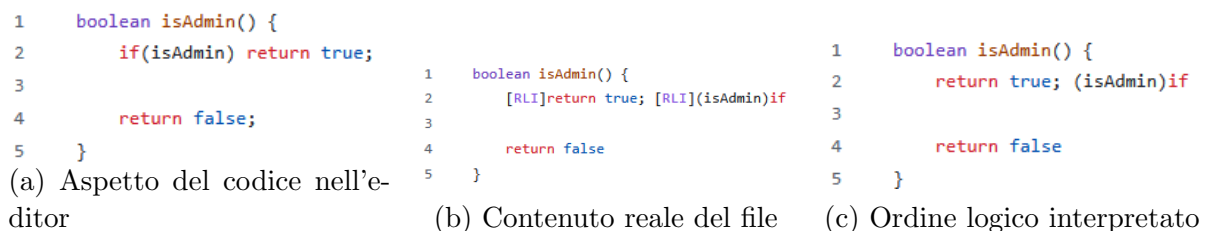


Figura 2: Codice nascosto nel pacchetto npm *rand-user-agent*.

Crediti: <https://www.aikido.dev/blog/catching-a-rat-remote-access-trojan-rand-user-agent-supply-chain-compromise>

Figura 3: Esempi di attacchi *Trojan Source*



Il risultato è che una riga di codice malevola può essere nascosta o fatta apparire come un commento innocuo scambiando l'ordine delle parole, ingannando un revisore umano. Questa vulnerabilità è presente in quasi tutti i linguaggi di programmazione moderni e, al momento della sua scoperta, era efficace sui più diffusi editor di codice e piattaforme come *VS Code*, *GitHub* e *GitLab* [4].

Un'altra forma di attacco HUT è l'attacco omoglifo, una tecnica di inganno visivo che sfrutta l'esistenza di caratteri Unicode che, pur apparendo identici, vengono codificati diversamente del compilatore (es. la lettera 'a' latina, codificata come U+0061, e la 'a' cirillica, codificata come U+0430).

Il meccanismo è il seguente:

1. Inserimento del carattere omoglifo: Un aggressore definisce una funzione o una variabile malevola utilizzando un nome che contiene un carattere omoglifo, rendendola visivamente identica a una funzione legittima. Ad esempio, una funzione dannosa chiamata `is_valid()` (con la 'a' cirillica) è indistinguibile dalla funzione sicura `is_valid()`.

2. Esecuzione inconsapevole: Uno sviluppatore che utilizza la libreria infetta può chiamare la funzione `malevola` senza accorgersene. Durante la revisione del codice, l'inganno è molto difficile da notare, poiché il nome della funzione appare visivamente corretto.

In sostanza, l'attacco omoglifo è una forma di “typosquatting” che avviene direttamente all'interno del codice sorgente, invece che a livello di nomi di pacchetti. Questo lo rende una minaccia più furtiva e difficile da individuare durante il normale processo di sviluppo e revisione [6].

Per esemplificare una manomissione tramite HUT, si consideri lo scenario illustrato nella Figura 4. Le immagini mostrano la schermata di una Pull Request (PR) che, a prima vista, sembra contenere solo modifiche minori e legittime. In realtà, nella Figura 4, la chiamata `ask_personal_name` a riga 24 contiene una ‘a’ cirillica che permette al programma di non chiamare la funzione legittima a riga 5 ma una malevola, definita in una dipendenza o in un altro file.

Questo esempio dimostra come gli aggressori possano offuscare il proprio attacco sfruttando le interfacce di code review e i limiti della percezione umana, trasformando il revisore nel punto debole della catena di sicurezza.

2.3 Gap da Colmare

Questo studio si vuole focalizzare su una nuova emergente classe di attacchi che non sfruttano falle o bug software ma gli sviluppatori stessi. Studi come “Trojan Source” [5] hanno dimostrato come la codifica Unicode possa essere usata per creare codice il cui aspetto maschera una logica malevola.

Nonostante la consapevolezza di queste minacce, gli strumenti di difesa attuali sono insufficienti: i tool SAST sono inefficaci, poiché esaminano la struttura logica del codice, ovvero l'Abstract Syntax Tree (AST), ma non la loro rappresentazione grafica.

Allo stesso modo, le piattaforme di sviluppo come *GitHub* hanno implementato contro-misure parziali, che però non coprono tutte le varianti di attacco e non sono sempre attive.

Queste carenze, unite alla mancanza di dataset pubblici di attacchi riconducibili alle tipologie BST e HUT, ha reso necessario lo sviluppo di una nuova metodologia di analisi basata sull'analisi storica dei progetti. La creazione di un dataset con vulnerabilità iniettate è stata un passo imprescindibile per poter testare e dimostrare l'efficacia di tale approccio.

3 Approccio Metodologico

Questa sezione presenta l'approccio metodologico adottato in questa tesi. La prima fase ha riguardato la costruzione di un opportuno dataset (Sezione 3.1), mentre la seconda fase si è concentrata sulla definizione della metodologia di analisi del codice sorgente

▼ 4 hut_test_attack.java

@@ -2,7 +2,7 @@

2 2

3 3 public class hut_test_attack {

4 4

5 - private void ask_name(Scanner scanner) {

5 + private void ask_personal_name(Scanner scanner) {

6 6 System.out.print("What is your name? ");

7 7

8 8 String name = scanner.nextLine();

9 9 System.out.println("Hello, " + name + "!");

10 10 }

11 11

12 12 private void ask_age(Scanner scanner) {

13 13 System.out.print("What is your age? ");

14 14

15 15 int age = scanner.nextInt();

16 16 System.out.println("You are " + age + " years old.");

17 17 }

18 18

19 19 public static void main(String[] args) {

20 20 Scanner scanner = new Scanner(System.in);

21 21

22 22 hut_test_attack instance = new hut_test_attack();

23 23

24 - instance.ask_name(scanner);

24 + instance.ask_personal_name(scanner);

25 25 instance.ask_age(scanner);

26 26

27 27 scanner.close();

Figura 4: Pull Request di un repository *GitHub*

(Sezione 3.2).

3.1 Costruzione di un Dataset contenente Vulnerabilità

Al fine di condurre un'analisi efficace, si è resa necessaria la creazione di un dataset contenente progetti che includessero vulnerabilità di tipo BST e HUT. Questo passaggio si è rivelato una scelta obbligata, poiché l'attuale letteratura scientifica non mette a disposizione dataset pubblici utili per l'indagine prevista in questo lavoro di tesi.

La generazione del dataset contenente le vulnerabilità ha richiesto la selezione di repository pubblici e la successiva iniezione di una backdoor.

Il processo implementato ha permesso di simulare l'azione di un attaccante verso una specifica repository in un ambiente controllato. Il codice compromesso non è stato in nessun modo proposto sui repository originali, rimanendo disponibile solo localmente.

3.1.1 Raccolta

Partendo da una lista più ampia di progetti JavaScript popolari, sono stati selezionati 91 repository che soddisfacevano pienamente i seguenti criteri di inclusione:

1. **Numero minimo di Tag:** ogni repository deve avere almeno 10 tag, quindi almeno 10 release di versioni successive che seguano lo standard del *semantic versioning* [9].
2. **Dimensione del Progetto:** In base a metriche quantitative e qualitative, i 91 repository del nostro studio sono stati suddivisi in due categorie:
 - **Progetti ‘Grandi’** (61 repository). Progetti maturi che soddisfano i seguenti criteri: un team con oltre tre contributori attivi, una cronologia estesa con centinaia di rilasci e indicatori di manutenzione frequente.
 - **Progetti ‘Piccoli’** (30 repository). Progetti ad autore singolo, le cui caratteristiche distintive sono uno stile di sviluppo uniforme e un ciclo di vita software gestito individualmente.

La lista dei repository *GitHub* presi in esame è disponibile al link https://github.com/marchacio/BST_HUT_analyzer/blob/main/run/const/repo_list.py

3.1.2 Iniezione della Backdoor

Per velocizzare il processo di creazione del dataset, è stata implementata una procedura automatizzata per iniettare una backdoor nel repository in esame.

Inizialmente il processo identifica l'ultima versione stabile del repository, ovvero l'ultimo tag. Successivamente, seleziona casualmente un file di codice sorgente dal progetto (escludendo file minificati, di configurazione o di test) e vi aggiunge una backdoor nascosta tramite i paradigmi offensivi BST e HUT.

Infine, la modifica viene salvata creando un nuovo commit e contrassegnando questa versione compromessa con un tag speciale denominato “TEST”.

Questa procedura è stata implementata attraverso uno script in Python ed è stata ripetuta per tutti i 91 progetti in analisi. Per rendere la creazione del dataset più realistica, si è scelto di iniettare una backdoor così strutturata:

```
1 var https = require('https');
2 https.get('https://www.fakemaliciouswebsite.it/fake-
  vulnerability-download',
3   res => res.forEach(
4     chunk => eval(chunk.toString())));
```

Il suo funzionamento prevede l'import del modulo *https* di *Node.js* alla riga 1; successivamente (righe 2-4), viene effettuata una richiesta GET a un server remoto e il contenuto della risposta viene eseguito direttamente tramite `eval()`, completando così l'attacco. Inoltre, tale codice viene offuscato tramite la funzione `fromCharCode()`, una tecnica comune per nascondere stringhe malevole. Il risultato finale sarà così composto:

```
1 eval(String.fromCharCode(118,97,114, ... , 41,41,59));
```

3.2 Metodologia

L'approccio per la costruzione dello strumento di analisi si fonda principalmente su un concetto: le vulnerabilità come BST e HUT vengono introdotte in un preciso istante della vita di un software e lasciano una traccia misurabile. Di conseguenza, il rilevamento più efficace non deriva da un'ispezione statica di una singola versione del codice, ma da un'analisi storica della sua evoluzione.

Il principio operativo è quello di trattare la sequenza di release di un repository Git (identificate dai tag) come una serie temporale nella quale, per ogni release, lo strumento calcola un "profilo metrico" di ciascun file sorgente.

La vulnerabilità non viene quindi rilevata dal valore assoluto di una metrica, ma dalla presenza di anomalie nella sua variazione tra una versione e quella immediatamente successiva. Una crescita improvvisa e statisticamente significativa del valore di una metrica per un determinato file è un segnale d'allarme, suggerendo una potenziale manomissione avvenuta in quella specifica transizione.

La gestione dei falsi positivi rappresenta una delle sfide principali di questo approccio. Dato che anche modifiche legittime e su larga scala possono causare picchi metrici, diventa fondamentale calibrare le soglie di allarme per minimizzare i falsi positivi. L'obiettivo è garantire che solo le deviazioni più sospette vengano segnalate, distinguendole efficacemente dalla normale evoluzione del software.

4 Studio delle Metriche

Questa sezione presenta il processo logico di selezione e validazione delle metriche usate per rilevare gli attacchi di tipo BST e HUT, evidenziando non solo le tecniche che si sono dimostrate efficaci, ma anche quelle che sono state scartate.

4.1 Analisi e Invalidazione delle Metriche Generiche

La fase iniziale della ricerca si è concentrata sull'applicazione di metriche standard, ampiamente utilizzate nella letteratura di ingegneria del software per valutare la qualità, la manutenibilità e l'evoluzione dei progetti. L'ipotesi di partenza era che l'introduzione di un attacco nel codice sorgente potesse generare perturbazioni misurabili anche a un livello macroscopico. Tuttavia questa ipotesi si è rivelata infondata per la tipologia di attacchi considerata.

Le prime metriche analizzate sono state quelle relative alla dimensione del codice:

- **Linee di Codice (LOC):** Utile per tracciare la crescita di un progetto, ma inefficace nel nostro contesto. Un attacco BST o HUT può essere strutturato su una singola riga di codice, anche modificandone una esistente. In un progetto da 50.000 LOC, l'aggiunta o la modifica di una riga rappresenta una variazione dello 0.002%, un segnale indistinguibile dal “rumore” generato dalla normale attività di sviluppo.
- **Dimensione in Byte del Codice:** Sebbene più sensibile del LOC, anche questa metrica è facilmente ingannata. Una modifica legittima come l'aggiunta di una dipendenza può aumentare la dimensione del progetto di svariati kilobyte o megabyte, mascherando completamente l'aumento di pochi byte o kilobyte dovuto all'inserimento di un payload malevolo. Inoltre, la sua dipendenza dalla verbosità del linguaggio e dalla formattazione la rende un indicatore instabile.

Un secondo approccio ha riguardato l'analisi dei metadati e dei pattern temporali del repository, ipotizzando che l'azione di un attaccante potesse deviare dalle normali routine di sviluppo.

- **Frequenza e Orari dei Commit:** Si è ipotizzato che una modifica malevola potesse avvenire in orari inusuali rispetto alla media. Questa metrica si è dimostrata inaffidabile: nei progetti open-source internazionali si possono avere contributori da fusi orari diversi e che quindi operano sul codice a tutte le ore del giorno. Inoltre, un attaccante può facilmente manipolare la data di un commit per farlo apparire normale agli occhi di un revisore.
- **Analisi delle Release:** L'analisi del tipo di release (major, minor, patch), della loro frequenza e degli autori si è rivelata inutile. Un attaccante può mascherare l'iniezione di codice malevolo all'interno di qualsiasi tipo di release ed adattandosi al ritmo di sviluppo del progetto.

Constatata l'inefficacia degli indicatori macroscopici, l'attenzione si è spostata su metriche più sofisticate, in grado di analizzare la struttura intrinseca del codice.

- **Complessità Ciclomatica:** Questa metrica quantifica la complessità del flusso di controllo di un programma ed è eccellente per identificare codice intricato. Tuttavia, un payload può essere molto semplice ed aggiunge al più un singolo nodo al grafo di controllo del programma. Inoltre, gli attacchi studiati in questa tesi non sfruttano la complessità logica, ma l'offuscamento lessicale e visivo.
- **Entropia di Shannon:** L'entropia misura quantità di informazione contenuta in un flusso di dati ed è spesso utilizzata con successo per rilevare codice compresso o crittografato, che tende ad avere un'entropia molto alta. Nel nostro caso,

questa metrica fallisce per ragioni opposte: un attacco BST inietta una sequenza estremamente lunga e ripetitiva del carattere spazio, la quale ha un'entropia vicina allo zero. L'aggiunta di questa sequenza ad un file di codice sorgente ne abbassa l'entropia media, un segnale che potrebbe essere interpretato erroneamente. Per gli attacchi HUT, l'introduzione di pochi caratteri Unicode in un file di centinaia o migliaia di righe è una perturbazione troppo piccola per essere rilevata da una misura calcolata sull'intero file.

Limiti dell'Analisi SAST Infine, è stato valutato l'impiego di tool *Static Application Security Testing* (SAST). Questi strumenti cercano pattern di codice corrispondenti a vulnerabilità note ma questo approccio presenta due problemi nel contesto da noi analizzato:

1. **Offuscamento dell'Attacco:** Gli attaccanti sviluppano costantemente nuove tecniche di offuscamento proprio per eludere le ricerche dei tool SAST.
2. **Contesto:** Il problema fondamentale è che i tool SAST analizzano l'AST del codice ma non sono progettati per vedere e valutare il codice come un essere umano. Di conseguenza, non possiedono meccanismi per rilevare anomalie nella distribuzione degli spazi bianchi, né per identificare l'uso ingannevole di caratteri omoglifi o di controllo della direzionalità del testo, che sono le basi degli attacchi BST e HUT.

4.2 Sviluppo e Validazione di Metriche Specifiche

Poiché le metriche generiche della Sezione 4.1 si sono rivelate inefficaci, è stato necessario cambiare approccio. L'analisi si è quindi concentrata sulla creazione di indicatori specifici per gli attacchi BST e HUT, esaminando il codice a livello di singola riga o carattere anziché di file. Infatti, il *modus operandi* di un attacco BST è l'iniezione di un numero massivo di spazi bianchi per spingere il payload fuori dalla vista dell'editor del programmatore umano; Questo comportamento lascia tracce statistiche evidenti.

- **Lunghezza Massima della Riga (MLL):** Un effetto collaterale diretto dell'attacco è la creazione di una riga di codice estremamente lunga. Tracciare la lunghezza massima di una riga per ogni file tra le varie versioni è un metodo efficace. Ad esempio, una riga che passa da 80 caratteri a 2080 è un'anomalia impossibile da ignorare. Il limite di questa metrica emerge nei file già caratterizzati da righe lunghe, come i file minificati.
- **Rapporto Spazi Bianchi/Caratteri Stampabili (W/P Ratio):** Tra tutte le metriche analizzate, questa si è rivelata la più potente e diretta per l'identificazione di attacchi BST. Il suo fondamento logico è molto semplice: ogni file di codice possiede un equilibrio relativamente stabile tra i caratteri che ne costituiscono la logica e gli spazi bianchi utilizzati per la formattazione e la leggibilità. Un attacco BST altera in modo inequivocabile questo equilibrio, aggiungendo un numero significativamente grande di caratteri vuoti.

Durante lo studio di questa metrica, sono state considerate due possibili formulazioni:

1. **Proporzione sul totale:** La prima opzione calcola la percentuale di spazi bianchi rispetto al numero totale di caratteri nel file.

$$Ratio_A = \frac{W}{P + W}$$

Dove W è il numero di spazi bianchi e P è il numero di caratteri stampabili. Ipotizziamo di aver un file con 800 caratteri e 200 spazi bianchi che subisce un attacco BST con l'aggiunta di 3000 spazi e un payload di 500:

– *Scenario Normale:*

$$Ratio_A = \frac{200}{800 + 200} = 0.2 \quad (20\%)$$

– *Scenario Attaccato:*

$$Ratio_A = \frac{200 + 3000}{(800 + 500) + (200 + 3000)} = 0.711 \quad (71.1\%)$$

Questa metrica funziona ed è intuitiva, ma il segnale è matematicamente smorzato dall'aumento del denominatore.

2. **Rapporto diretto (Scelta Adottata):** La seconda opzione calcola il rapporto diretto tra spazi bianchi e i soli caratteri stampabili.

$$Ratio_B = \frac{W}{P}$$

– *Scenario Normale:*

$$Ratio_B = \frac{200}{800} = 0.25 \quad (25\%)$$

– *Scenario Attaccato:*

$$Ratio_B = \frac{200 + 3000}{800 + 500} = 2.46 \quad (246\%)$$

Questa metrica produce un segnale molto più “esplosivo”, con un valore che aumenta di circa 10 volte nello scenario di esempio.

Si è quindi scelto di adottare la seconda formulazione ($Ratio_B$) che massimizza la sensibilità della metrica per il rilevamento di anomalie. Questo crea un segnale molto più netto ed isolabile, rendendo il sistema di rilevamento più affidabile.

Gli attacchi HUT sono più subdoli perché manipolano la rappresentazione grafica del testo e il loro rilevamento richiede un'ispezione a livello di singolo carattere, alla ricerca di specifici codici Unicode.

- **Conteggio di Caratteri Omoglifi:** La strategia consiste nel creare una lista di omoglifi noti (es. caratteri cirillici, greci o latini che assomigliano a caratteri ASCII) e scansionare il codice sorgente alla ricerca della loro presenza. Sebbene l'uso occasionale di caratteri non-ASCII possa essere legittimo (es. in stringhe o commenti), la loro presenza all'interno di identificatori di variabili o nomi di funzioni è altamente sospetta. Uno scenario di attacco potrebbe essere:

```

1 // Funzione legittima
2 function isAdmin(user) { /* ... */ }

1 // Funzione malevola con omoglifo (con la 'А' cirillica)
2 function isAdmin(user) { /* codice malevolo */ }

```

Infatti, un attaccante potrebbe sostituire una chiamata alla funzione legittima con quella malevola e la modifica sarebbe quasi invisibile in una code review. Il metodo sviluppato si basa sul conteggio di questi caratteri sospetti, segnalando i file che ne presentano una o più occorrenze o un aumento.

- **Conteggio di Caratteri Invisibili:** Questa è la metrica più specifica e diretta per gli attacchi di tipo *Trojan Source* [5]. La procedura consiste nello scansionare il codice alla ricerca di specifici caratteri Unicode che alterano la logica di interpretazione del sorgente. Tra questi, i più pericolosi sono i caratteri *BiDi* [12], come 'U+202E' (Right-to-Left Override), il cui scopo originale è di includere gli alfabeti che si leggono da destra verso sinistra come l'arabo e l'ebraico. Lo strumento sviluppato mantiene un contatore di questi caratteri e ne segnala la presenza in qualsiasi punto del codice, poiché il loro uso in un file di programmazione è quasi sempre illegittimo.

In conclusione, il percorso metodologico ha dimostrato che per contrastare minacce specifiche e non convenzionali come BST e HUT le metriche software generiche sono insufficienti. È stato necessario un approccio mirato, basato su una profonda comprensione delle tecniche di attacco, che ha portato allo sviluppo di indicatori a grana fine.

5 Sviluppo della Soluzione

Questa sezione descrive l'architettura, l'implementazione e l'evoluzione dello strumento software sviluppato per l'analisi e il rilevamento delle vulnerabilità BST e HUT. Il tool, denominato *BST_HUT_analyzer* e disponibile su *GitHub* all'indirizzo https://github.com/marchacio/BST_HUT_analyzer, è scritto in Python e implementa le metriche identificate nella sezione 4.2.

Lo strumento è stato concepito per analizzare in modo sistematico la cronologia di un repository Git, individuando anomalie a livello delle singole release (tag).

Nello specifico, verranno approfonditi i seguenti aspetti chiave del processo di sviluppo: Architettura del Software (Sezione 5.1), Sistema di Configurazione (Sezione 5.2), Componenti di Analisi (Sezione 5.3), Ottimizzazione delle Prestazioni (Sezione 5.4) e Automazione del Processo (Sezione 5.5)

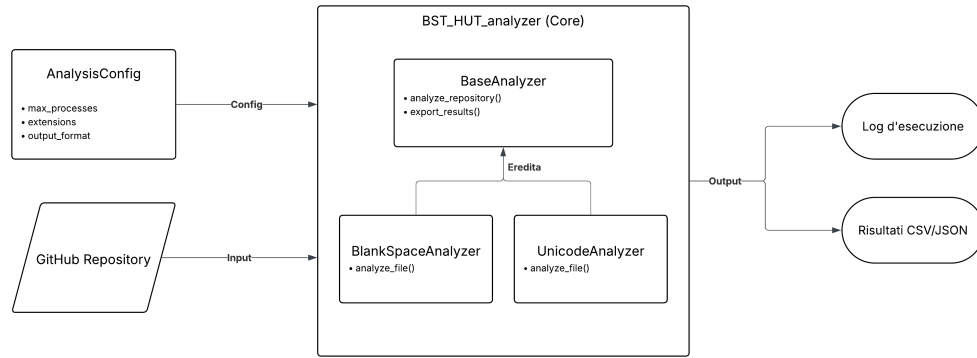


Figura 5: *Pseudo Class Diagram* del progetto

5.1 Architettura del Software

Come mostra la Figura 5, il fulcro del progetto è la classe base astratta **BaseAnalyzer**, definita nel file `core/analyzer.py`. Questa classe agisce come un “template” per la creazione di nuovi moduli di analisi, definendo un’interfaccia comune e gestendo le operazioni di base.

Le responsabilità della classe **BaseAnalyzer** sono:

- **Gestione dell’Analisi:** Il metodo `analyze_repository` itera su tutti i tag di un repository, ordinandoli cronologicamente. Per ogni tag, effettua il checkout della versione corrispondente del codice e avvia il processo di analisi del tag selezionato.
- **File Discovery:** Attraverso il metodo `_get_files_for_extension`, il sistema ricerca ricorsivamente tutti i file con un’estensione specifica (es. `.js`), escludendo al contempo le directory non pertinenti (come `node_modules` o `.git`) per ridurre il rumore e l’overhead.
- **Gestione dei Risultati:** La classe base definisce una struttura dati standard per i risultati, utilizzando le dataclass `FileAnalysisResult` e `TagAnalysisResult` per aggregare metriche, anomalie e metadati.
- **Esportazione Standardizzata:** Il metodo `export_results` gestisce il salvataggio dei dati raccolti in formati quali JSON e CSV, delegando l’implementazione specifica del formato CSV alle sottoclassi.

Questo design consente di estendere le capacità del tool introducendo nuovi analizzatori (es. per una nuova classe di vulnerabilità) senza modificare l’architettura centrale, ma semplicemente creando una nuova classe che eredita **BaseAnalyzer**.

5.2 Sistema di Configurazione Centralizzato

È stato implementato un sistema di configurazione centralizzato per gestire il comportamento del tool in modo flessibile e senza la necessità di modificare il codice sorgente. La soluzione si basa sulla classe **AnalysisConfig**, definita nel file `config.py`, che raggruppa

tutti i parametri che influenzano l'esecuzione come `supported_extensions` per configurare le estensioni dei file da esaminare o `filter_dirs` ignorare cartelle e sottocartelle, come ad esempio `node_modules` o `dist`.

Riguardo le prestazioni, il parametro `max_processes` consente di regolare il numero di processi paralleli da utilizzare, permettendo di ottimizzare le prestazioni in base all'hardware disponibile. Infine, per l'output, si può scegliere il formato dei risultati (`output_format`) tra CSV e JSON e definire il percorso dei file di log (`log_file`). Una caratteristica chiave della classe `AnalysisConfig` è il metodo `__post_init__`, che imposta valori di default intelligenti qualora non vengano forniti esplicitamente.

5.3 Componenti di Analisi Implementati

Sfruttando l'architettura astratta di base, sono stati implementati due analizzatori concreti: `BlankSpaceAnalyzer`, localizzato in `core/blank_space_analyzer.py`, che implementa le metriche discusse nella Sezione 4.2 per l'identificazione di attacchi BST.

Allo stesso modo, `UnicodeAnalyzer`, situato in `core/unicode_analyzer.py`, è stato sviluppato per contrastare gli attacchi HUT. Per migliorare l'efficienza, l'analisi non viene eseguita su ogni singolo carattere del file ma viene calcolata la frequenza di ogni carattere unico tramite `collections.Counter` e l'analisi viene poi eseguita solo su questo set ridotto di caratteri.

5.4 Ottimizzazione delle Prestazioni

Nelle fasi iniziali dello sviluppo, l'analisi dei file veniva eseguita in modo puramente sequenziale. Sebbene funzionale, questo approccio si è rivelato un collo di bottiglia durante l'analisi di repository di grandi dimensioni, infatti il tempo di analisi per un singolo repository poteva richiedere diverse ore, limitando la fattibilità di esperimenti su larga scala.

Per superare questo limite, è stato introdotto il supporto alla parallelizzazione grazie alla libreria *multiprocessing* di Python.

Tuttavia, durante la fase di testing è emerso un importante compromesso legato a questa ottimizzazione. Per i repository di dimensioni medio-piccole, l'overhead computazionale associato alla creazione, alla sincronizzazione e alla chiusura della pool di processi si è rivelato superiore al guadagno ottenuto. In questi scenari, l'analisi sequenziale è risultata di fatto più veloce ed efficiente.

Lo strumento è stato configurato in modo tale da permettere all'utente di scegliere la strategia più adatta al contesto: la modalità parallela garantisce la scalabilità necessaria per affrontare progetti di livello enterprise, mentre quella sequenziale rimane l'opzione più performante per analisi rapide su progetti più contenuti.

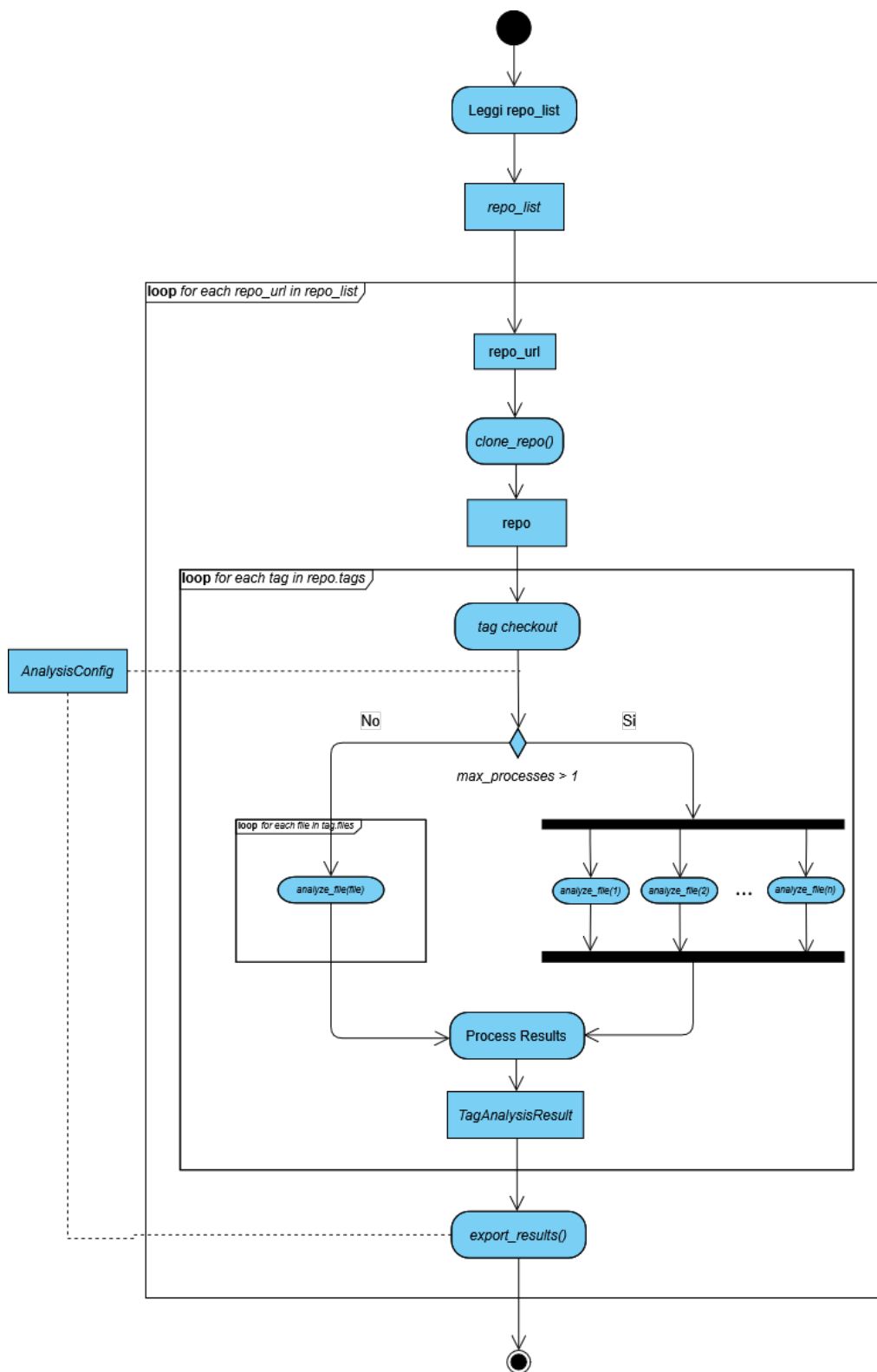


Figura 6: Activity Diagram dell'analisi del dataset

5.5 Automazione del Processo di Analisi

Per garantire la ripetibilità e la coerenza nella raccolta dei dati su larga scala, il processo di analisi dei repository è stato completamente automatizzato tramite un sistema di script dedicati che seguono le indicazioni specificate nel *activity diagram* mostrato nella Figura 6.

Il sistema si basa su due componenti principali: una lista centralizzata di repository target, definita nel file `run/const/repo_list.py`, e una serie di script di esecuzione (`original_blank_space_run.py`, `manipulated_blank_space_run.py`, `unicode_run.py`) che gestiscono l'intero flusso di lavoro.

Ogni script itera sulla lista dei repository, configura l'analizzatore specifico (`BlankSpaceAnalyzer` o `UnicodeAnalyzer`), clona il codice sorgente, esegue l'analisi, salva i risultati in formato CSV e un file di log per ripercorrere l'andamento dell'esecuzione.

6 Risultati

In questa sezione vengono presentati i risultati ottenuti dall'applicazione del tool sul dataset descritto in Sezione 3.1. In particolare, Sezione 6.1 presenta l'efficacia del rilevamento degli attacchi BST, mentre Sezione 6.2 si focalizza sugli attacchi HUT. Per entrambe le tipologie vengono analizzati i risultati ottenuti dell'analisi di quattro progetti rappresentativi di metodologie di sviluppo comuni nel contesto open-source.

Nel dettaglio due dei progetti (*Angular* [3] e *Vue* [11]) hanno un grande numeri di contributori, un'intensa attività di sviluppo (più di 50 commit a settimana), e una grande quantità di codice (1.000.000 LoC in media).

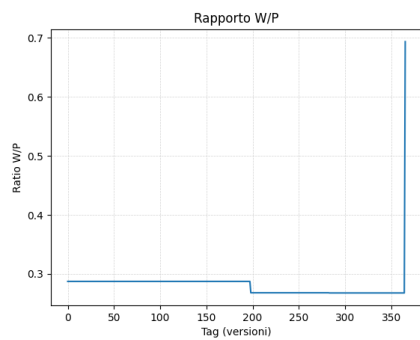
Diversamente, due dei progetti selezionati (*Superstruct* [7] e *Redom* [10]) hanno un ridotto numero di contributori attivi (3 o meno), un'attività di sviluppo ridotta (meno di 10 commit a settimana), e una ristretta quantità di codice (7500 LoC in media). Infine, Sezione 6.3 analizza le cause dei falsi positivi ottenuti durante gli esperimenti condotti.

6.1 Efficacia nel Rilevamento degli Attacchi BST

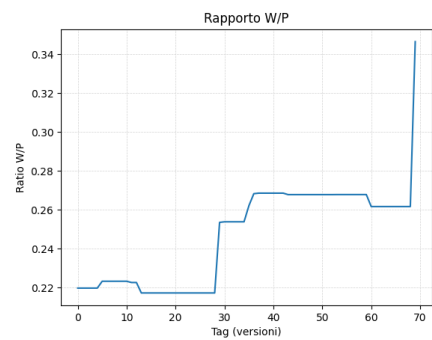
La soluzione proposta si è dimostrata efficace nell'identificare la presenza di attacchi BST. In particolare, Figura 7, mostra chiaramente come il W/P ratio si rende efficace sia nel modellare il ciclo di vita del software, sia a mostrare eventuali anomalie.

In particolare, nel caso di processi di sviluppo che causano un andamento non costante di questa metrica, come *Vue*, è possibile identificare chiaramente l'anomalia causata dall'attacco. Infatti, in questo caso *Vue* mostra un primo incremento del 21% nella metrica tra le realease *2.6.14* e *2.7.0-alpha.1* (rispettivamente la 28^a e la 29^a release per il file preso in analisi).

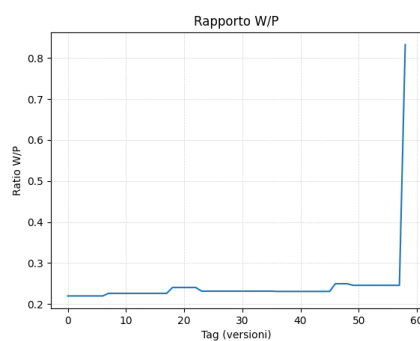
Questo incremento è dovuto all'introduzione di nuove funzionalità e modifiche strutturali significative nella minor release, che hanno portato a un aumento legittimo degli spazi



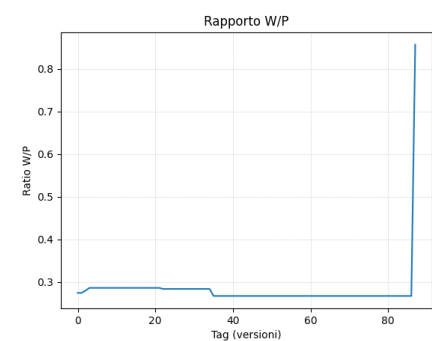
(a) *Angular*



(b) *Vue*



(c) *Redom*



(d) *Superstruct*

Figura 7: Quattro grafici che mostrano l'andamento temporale della metrica W/P Ratio dei repository presi in esame.

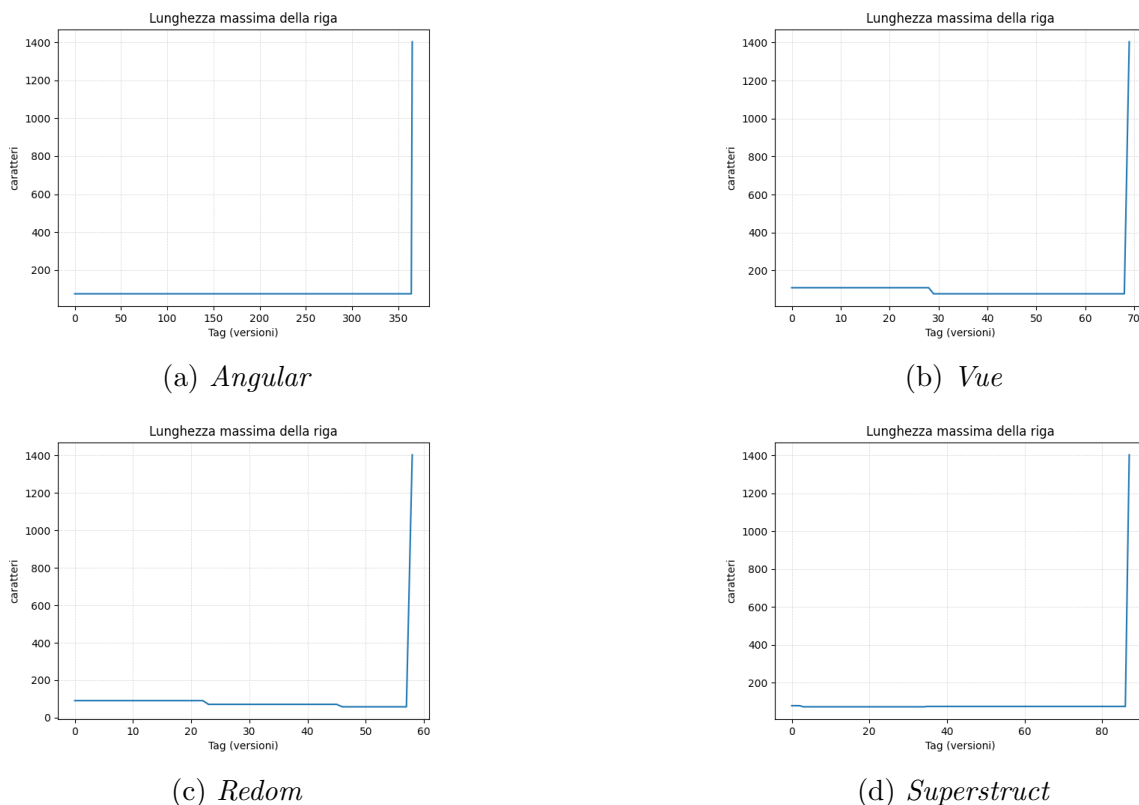


Figura 8: Quattro grafici che mostrano l’andamento temporale della metrica MLL dei repository presi in esame.

bianchi e della formattazione nel codice. Tuttavia l’incremento risulta meno evidente di quello causato dall’inserimento del payload (35%). Questa metrica si dimostra particolarmente efficace nel caso di un processo di sviluppo intenso come quello di *Angular*, dove il grande numero di release (>300) rende ancora più evidente l’anomalia.

La metrica riesce ad essere efficace anche per i progetti selezionati per rappresentare metodi di sviluppo meno intensi, *Superstruct* e *Redom*. In particolare, su questi progetti si nota un andamento meno costante nel periodo precedente all’inserimento del payload, probabilmente dovuto a cambiamenti meno costanti nel tempo e quindi maggiormente visibili (es., un singolo sviluppatore inserisce notevoli quantità di codice dopo lunghi intervalli di tempo, questo viene catturato in modo più visibile dalla metrica).

Figura 8 mostra il comportamento della metrica MLL. È possibile notare come anch’essa sia capace di catturare la presenza del payload malevolo all’interno del codice. Tuttavia non si dimostra particolarmente rappresentativa del ciclo di vita del software, infatti mantiene un valore quasi costante per tutti i progetti analizzati.

Questo dato è fortemente influenzato dalle pratiche standard comuni tra gli sviluppatori, che tendono a non superare gli 80 caratteri per linea [8]. Inoltre, come meglio dettagliato in seguito, questa metrica coglie una caratteristica istantanea del codice, che potrebbe essere alterata da cause comuni.

Un esempio di queste possibili alterazioni è il caso di funzioni lambda particolarmente complesse, che hanno la possibilità di essere scritte in linea nella maggior parte dei linguaggi di programmazione.

Tabella 1: Variazione delle metriche HUT per i file dei repository presi in esame.

Progetto	Tag	Conteggio Omoglifi	Caratteri Totali
<i>Angular</i>	...	0	...
	v14.2.9	0	3589
	v14.2.10	0	3589
	v14.2.11	0	3589
	TEST	4	4683
<i>Vue</i>	...	0	...
	v3.2.43	0	8732
	v3.2.44	0	8822
	v3.2.45	0	8822
	TEST	4	9916
<i>Redom</i>	...	0	...
	v3.25.0	0	3281
	v3.26.0	0	3281
	v3.27.0	0	3310
	TEST	4	4404
<i>Superstruct</i>	...	0	...
	v0.6.0	0	696
	v0.6.1	0	696
	v0.7.0	0	696
	TEST	4	1790

6.2 Efficacia nel Rilevamento degli Attacchi HUT

La soluzione proposta si è dimostrata efficace nell'identificare la presenza di attacchi HUT. La nostra analisi ha rivelato che i caratteri omoglifi sono estremamente rari all'interno dei progetti esaminati: su un totale di 41.903 tag e 91 repository, è stata riscontrata la loro presenza in sole 156 versioni, sparse tra 10 progetti, pari allo 0.3% dei tag analizzati. Questa caratteristica permette di identificare in modo efficace attacchi HUT osservando in modo diretto la presenza o l'inserimento di caratteri omoglifi. Prendendo in analisi i quattro progetti già considerati per gli attacchi BST, Tabella 1 mostra che i file nei quali è stato inserito il payload malevolo non contengono nessun carattere omoglifo in tutto il loro ciclo di vita.

Tuttavia nella release "TEST", in cui è stata aggiunta una backdoor nascosta tramite HUT, si può notare un incremento sospetto e anomalo del numero di caratteri omoglifi e del totale, segno inconfondibile di una manomissione da parte di un attaccante.

Sebbene in questo specifico scenario di test le due metriche varino insieme, è fondamentale sottolineare che la dimensione del file non è un indicatore affidabile per gli attacchi HUT. Un aggressore potrebbe sostituire un solo carattere legittimo con un suo omoglifo senza alterare la lunghezza totale del codice, rendendo l'attacco invisibile a un'analisi basata sulla sola dimensione, pertanto il fatto che il numero di caratteri totali sia rimasto costante per numerose versioni non è una condizione sufficiente.

Il vero segnale d'allarme è la natura binaria (presenza/assenza) dei caratteri sospetti che

la rende un segnale d'allarme ad alta fedeltà che permette di isolare la minaccia con precisione.

6.3 Analisi dei Falsi Positivi

Un aspetto fondamentale per valutare l'efficacia delle metriche selezionate è la capacità di non generare un numero eccessivo di falsi allarmi. Per identificare le cause dei falsi positivi, lo strumento è stato eseguito sul dataset escludendo la versione in cui è stato inserito il payload malevolo. Ogni anomalia è stata considerata un falso positivo e analizzata per classificarne l'origine.

Nel caso di attacchi BST la causa principale risiede nella presenza di file minificati all'interno del progetto. Quando uno sviluppatore crea una nuova versione del progetto aggiungendo asset, file di test o file di configurazione minificati, le metriche selezionate vengono invalidate. In particolare, il W/P Ratio viene influenzato dalla rimozione degli spazi bianchi applicata dal processo di minificazione, mentre la lunghezza massima della riga viene resa superflua dalla rimozione degli ritorni a capo presenti nel codice.

È giusto evidenziare che il processo di minificazione influenza anche la riuscita dell'attacco, infatti esso cambia drasticamente la visualizzazione del codice. Una volta minificato, il payload potrebbe apparire chiaramente allo sviluppatore, oppure essere reso non comprensibile, a seconda del processo di minificazione utilizzato.

Tuttavia, la presenza di falsi positivi può essere mitigata con semplici modifiche all'approccio utilizzato. In dettaglio, l'attuale implementazione del tool applica un filtro per ignorare alcuni file in base alla loro estensione (es. `.min.js`, `.min.css`, `.map`) e per escludere dall'analisi alcune directory comunemente usate come destinazione dei processi di build o per librerie di terze parti (es. `dist`, `build`, `vendor`, `assets`) che non costituiscono superficie d'attacco per le tipologie di codice malevolo analizzate in questa tesi.

Allo stesso modo, possiamo identificare le cause della presenza di falsi positivi durante l'identificazione di attacchi HUT. In particolare, all'interno del nostro dataset abbiamo identificato tre cause rilevanti: simboli matematici, file di localizzazione e potenziali copia-incolla.

I primi sono presenti in particolari in progetti che contengono funzionalità legate a operazioni matematiche in cui gli sviluppatori hanno inserito commenti in cui descrivono le funzioni implementate. I secondi riguardano progetti in cui viene offerta la localizzazione (traduzione) del testo per alcuni componenti grafici, questo implica la presenza di caratteri con encoding differente. L'ultima causa è dovuta al copia-incolla da file in altri formati, presumibilmente PDF, che introducono caratteri non visibili.

Questi tipi di falsi positivi possono essere mitigati implementando un approccio legato all'AST del programma. Questo permetterebbe di focalizzare l'analisi solo su componenti ritenuti passibili di attacco, come i nomi di funzioni o variabili.

Un altro approccio potrebbe essere l'esclusione di alcuni file dall'analisi, con il compromesso che l'attacco potrebbe essere effettivamente contenuto al loro interno.

7 Discussione

Questa sezione finale ha lo scopo di interpretare i risultati ottenuti, contestualizzare i contributi di questa tesi nel panorama della sicurezza del software, analizzarne i limiti intrinseci e delineare le possibili direzioni per lavori futuri. L'analisi condotta ha dimostrato che, sebbene gli attacchi come BST e HUT mirino a superare la percezione umana, essi lasciano tracce evidenti, rilevabili attraverso un approccio mirato.

7.1 Interpretazione dei Risultati

I risultati presentati nella Sezione 6 confermano l'ipotesi centrale di questa tesi: un'analisi evolucionistica e differenziale basata su metriche specifiche è in grado di rilevare con successo attacchi progettati per essere invisibili all'occhio umano.

Per gli attacchi di tipo BST, la metrica W/P Ratio si è rivelata eccezionalmente efficace. Un progetto software, grande o piccolo che sia, tende a mantenere un relativo equilibrio tra codice e spazi bianchi ma l'iniezione di un attacco BST rompe drasticamente questa stabilità, causando un'impennata anomala del rapporto.

La metrica MLL agisce come un indicatore di conferma in quanto gli standard di codifica moderni raccomandano righe di codice di massimo 120 caratteri ed il passaggio ad una riga di oltre 1400 caratteri (come nell'attacco simulato) rappresenta un'anomalia statisticamente impossibile da ignorare in un file di codice "sano".

Per gli attacchi di tipo HUT, l'efficacia della metrica di OC e IC risiede nella loro "natura binaria": la loro apparizione in un file di codice, in particolare all'interno di nomi di funzioni o variabili, è quasi sempre un'anomalia. È vero che alcuni file potrebbero contenere legittimamente caratteri non-ASCII (come discusso nell'analisi dei falsi positivi), ma il principio dell'analisi storica rimane valido: l'improvviso aumento del loro numero all'interno del codice costituisce un campanello d'allarme da non sottovalutare.

7.2 Fragilità dell'Ecosistema di Sviluppo

Un punto cruciale emerso da questa tesi è la fragilità dell'ecosistema di sviluppo di fronte a queste tipologie di attacchi. Per verificare lo stato attuale delle difese a questi tipi di attacchi, è stata condotta un'analisi sulle principali piattaforme di hosting del codice al fine di valutare come gestiscono la visualizzazione del codice sorgente e se implementano contromisure contro l'offuscamento visivo.

I risultati, mostrati nella Tabella 2, rivelano una situazione disomogenea e frammentata. Infatti, anche una piattaforma matura come *GitHub* offre una protezione parziale: se da un lato il *soft-wrap* nelle Pull Request può mitigare un attacco BST, questa difesa non viene applicata quando si naviga il codice direttamente in un repository. In questa modalità, la barra di scorrimento orizzontale può addirittura scomparire dopo pochi secondi di inattività, rendendo un payload nascosto quasi impossibile da individuare.

Per gli attacchi HUT, la situazione è ancora più critica: il software segnala solo l'eventuale presenza di caratteri di controllo *BiDi*, ma non vi è alcun avviso contro l'uso di omoglifi o altri caratteri invisibili nel codice, che rappresentano il cuore di questa minaccia.

Tabella 2: Analisi delle contromisure contro attacchi invisibili nelle piattaforme di sviluppo.

Piattaforma	Mitigazione BST	Mitigazione HUT
<i>GitHub</i>	Parziale ¹	Parziale ²
<i>RubyGems</i>	Sì	No
<i>npm</i>	No	No
<i>NuGet</i>	No	No
<i>Yarn</i>	No	No
<i>PyPI</i>	N.D. ³	N.D. ³
<i>CocoaPods</i>	N.D. ³	N.D. ³
<i>Anaconda</i>	N.D. ³	N.D. ³

¹ Il wrap del testo è attivo nelle Pull Request, ma non nella visualizzazione standard dei file nel repository.

² Viene mostrato un avviso solo per i caratteri di controllo BiDi, ma non per gli omoglifi o altri caratteri invisibili.

³ La piattaforma non offre una visualizzazione del codice nativa e rimanda a repository esterni (tipicamente *GitHub*).

Altre piattaforme, come *npm*, *NuGet* e *Yarn*, non applicano alcuna forma di wrap, lasciando il revisore umano completamente esposto. Molti altri registri di pacchetti, come *PyPI* e *CocoaPods*, delegano interamente la visualizzazione del codice a *GitHub*, ereditandone di fatto le stesse vulnerabilità.

Questa diffusa mancanza di una difesa standardizzata e completa a livello di infrastruttura rafforza la necessità di strumenti di analisi specializzati.

7.3 Contributi della Tesi

Lo studio condotto offre diversi contributi al campo della sicurezza della SSC. In particolare, è stato dimostrato che le metriche software generiche sono inadeguate per rilevare attacchi basati sull'offuscamento visivo. Al contrario, è stato proposto e validato un approccio basato sull'analisi evolutiva di metriche a grana fine progettate appositamente per questi tipi di attacchi.

È stato inoltre sviluppato e rilasciato uno strumento open-source che implementa la metodologia proposta, *BST_HUT_analyzer*: Il tool è scalabile, configurabile e potenzialmente può essere utilizzato da maintainer di progetti per cercare manomissioni passate o per monitorare nuove contribuzioni.

Infine, è stato creato un dataset di 91 repository, sia originali che manipolati con finte backdoor, che può servire come base per futuri studi.

7.4 Limiti dello Studio

Nonostante i risultati positivi ottenuti, è fondamentale riconoscere i limiti di questo studio che non inficiano la validità dell’approccio, ma ne definiscono i confini e aprono la strada a ricerche future.

- **Natura del Dataset:** La validazione dello strumento e delle metriche si basa su un dataset in cui le vulnerabilità sono state iniettate artificialmente: per quanto questo approccio abbia permesso di creare un ambiente di test controllato e ripetibile, esso rappresenta un’astrazione di uno scenario reale. Infatti un attaccante potrebbe impiegare tecniche di offuscamento varie e distribuite su più commit per ridurre al minimo le anomalie statistiche, presentando una sfida più complessa rispetto all’attacco singolo simulato.
- **Contesto per Attacchi HUT:** Una limitazione dell’attuale implementazione per il rilevamento di attacchi HUT è la mancanza di sensibilità al contesto sintattico: lo strumento segnala la presenza di caratteri omoglifi o invisibili indipendentemente dalla loro posizione. Come discusso nell’analisi dei falsi positivi, il tool non distingue tra un carattere sospetto in un identificatore di funzione e lo stesso carattere in un commento o in una stringa di internazionalizzazione. Questa mancanza, che potrebbe essere superata integrando un’analisi dell’AST, rende necessario un intervento umano per validare gli allarmi.
- **Dipendenza dalla Cronologia:** L’efficacia della metodologia proposta è intrinsecamente legata al suo approccio di analisi storica, che rileva anomalie confrontando le diverse release di un progetto. Questa dipendenza da una cronologia pregressa costituisce un limite importante: il metodo necessita di un numero sufficiente di versioni “sane” per stabilire una base di partenza. Di conseguenza, nelle prime fasi di vita di un repository, lo strumento è poco efficace nel rilevare un attacco.
- **Focalizzazione sul Linguaggio di Programmazione:** Sebbene le metriche siano state concepite per essere applicabili a qualsiasi linguaggio di programmazione, il dataset e la validazione empirica si sono concentrati prevalentemente su repository JavaScript. I valori delle metriche possono variare notevolmente in base al linguaggio utilizzato per via di formattazioni e convenzioni adottate. Di conseguenza, l’efficacia e delle eventuali soglie di allarme del tool potrebbero richiedere una “ricalibrazione” per garantire la stessa affidabilità in altri ecosistemi di programmazione.

7.5 Lavori Futuri e Implicazioni Pratiche

L’analisi condotta e lo strumento sviluppato in questa tesi rappresentano una base di partenza per future ricerche e applicazioni pratiche nel campo della sicurezza della SSC. Le direzioni future possono essere suddivise in miglioramenti diretti dello strumento e in implicazioni più ampie per l’ecosistema di sviluppo.

La principale applicazione pratica di *BST_HUT_analyzer* è la sua integrazione nei sistemi di CI/CD. Lo strumento potrebbe essere configurato come uno step automatico che analizza ogni nuova pull request o commit ed invece di analizzare l’intera cronologia,

potrebbe confrontare le metriche dei file modificati con la loro versione precedente nel branch principale. Un'impennata anomala in una delle metriche chiave potrebbe bloccare automaticamente il processo di merge e notificare i maintainer, trasformando lo strumento in un sistema di difesa attivo.

Inoltre, per superare i limiti identificati e aumentare l'efficacia dello strumento, è stata stilata una lista di direzioni di ricerca future:

- **Analisi Contestuale tramite AST per Attacchi HUT:** Un'evoluzione cruciale è l'analisi contestuale del codice tramite l'AST. Questo metodo consente di capire dove un carattere omoglifo viene usato: se in un nome di variabile o funzione (minaccia critica) o in un semplice commento (bassa criticità). In questo modo, si abbattano drasticamente i falsi positivi e si forniscono ai revisori allarmi più precisi.
- **Approcci Basati su Machine Learning:** Per superare il rilevamento basato solo su variazioni improvvise, si potrebbe applicare il machine learning; addestrando un modello sulla cronologia di un progetto imparerebbe le sue fluttuazioni tipiche, diventando così capace di identificare attacchi più sofisticati e ben sviluppati.
- **Ampliamento del Dataset con Scenari di Attacco Complessi:** Il dataset di test potrebbe essere ampliato per includere scenari di attacco più sofisticati, come attacchi distribuiti su più file, attacchi che combinano tecniche BST e HUT, o backdoor che vengono attivate solo in combinazione con modifiche in altri punti del codice.

7.6 Conclusioni

Questa tesi ha affrontato il problema degli attacchi al codice sorgente, come i Blank Space Trojan (BST) e gli Hidden Unicode Trojan (HUT), che sfruttano i limiti della percezione umana per aggirare i processi di code review.

È stato dimostrato che le metriche software tradizionali sono inefficaci contro queste minacce e la soluzione proposta si basa su un'analisi evoluzionistica delle release di un progetto, utilizzando metriche a grana fine specificamente progettate per rilevare le anomalie introdotte da tali attacchi.

Pur mostrando un'elevata efficacia, l'approccio presenta margini di miglioramento come discusso alla sezione 7.5. Il passo successivo è l'integrazione del tool in una pipeline di CI/CD, trasformandolo in uno strumento di difesa vero e proprio che possa allertare i maintainer in tempo reale.

Infine, questo studio ribadisce il bisogno di rafforzare la fiducia nell'ecosistema open source e per farlo si necessitano nuovi strumenti che assistano l'uomo e aumentino la sicurezza dei prodotti informatici e delle loro dipendenze.

Riferimenti bibliografici

- [1] Qwiet AI. Github hijack: What you should know. <https://qwiet.ai/github-hijack-what-you-should-know/>.
- [2] Aikido security. <https://www.aikido.dev/>.
- [3] Angular. Angular github repository. <https://github.com/angular/angular>.
- [4] Cve-2021-42574. <https://nvd.nist.gov/vuln/detail/cve-2021-42574>.
- [5] Nicholas Boucher and Ross Anderson. Trojan Source: Invisible Vulnerabilities. In *32nd USENIX Security Symposium (USENIX Security 23)*, Anaheim, CA, August 2023. USENIX Association.
- [6] Cve-2021-42694. <https://nvd.nist.gov/vuln/detail/cve-2021-42694>.
- [7] Ianstormtaylor. Superstruct github repository. <https://github.com/ianstormtaylor/superstruct>.
- [8] Delano Oliveira, Reydne Santos, Fernanda Madeiral, Hidehiko Masuhara, and Fernando Castor. A systematic literature review on the impact of formatting elements on code legibility. *Journal of Systems and Software*, 203:111728, 2023.
- [9] Tom Preston-Werner. Semantic versioning 2.0.0. <https://semver.org/>.
- [10] Redom. Redom github repository. <https://github.com/redom/redom>.
- [11] Vuejs. Vue github repository. <https://github.com/vuejs/vue>.
- [12] Wikipedia. Bidi chars. https://en.wikipedia.org/wiki/Bidirectional_text.

Ringraziamenti

Desidero ringraziare calorosamente i miei relatori Giacomo Benedetti e Luca Caviglione per la loro costante guida, per la pazienza e gli insegnamenti trasmessi. Un ringraziamento speciale anche al Prof. Giovanni Lagorio per la grande disponibilità e il prezioso aiuto. Un'ultima enorme riconoscenza alla mia famiglia e ai miei amici più stretti, sempre al mio fianco.